

Generative Adversarial Networks

From VAEs to GANs: Theory, Proofs & Applications

COMPREHENSIVE LECTURE NOTES

Deep Learning and its Applications

Prof. Aditya Nigam

Manas Lab

Indian Institute of Technology Mandi

YouTube Channel: *Deep Learning for All at Manas Lab, IIT Mandi*

March 18, 2026

Contents

1	Variational AutoEncoder — Summary (Review)	3
1.1	Architecture Overview	3
1.2	The ELBO Objective	3
1.3	Loss Functions in VAE	4
2	Problems with the L_2/MSE Loss	5
2.1	The MSE Loss Function	5
2.2	The Perceptual Limitation of MSE	5
2.3	Connection Between MSE and Gaussian Likelihood	5
2.4	The Blurring Problem: Bimodal Distributions	6
3	KL Divergence: Properties and Limitations	7
3.1	Definition and Asymmetry	7
3.2	Forward vs. Reverse KL Divergence	7
3.3	Why VAEs Use the Reverse KL Divergence	7
3.3.1	Forward KL Divergence Analysis	7
3.3.2	Reverse KL Divergence Analysis	8
3.4	The Fundamental Problem with KL Divergence	9
4	Jensen–Shannon Divergence	10
4.1	Definition	10
4.2	Key Properties	10
4.3	Worked Example: Non-overlapping Distributions	10
5	From VAEs to GANs: Motivation and Architecture	11
5.1	Recap: The VAE Framework	11
5.2	The GAN Paradigm Shift	11
5.3	GAN Architecture	11
6	GAN Objective Function: Derivation	13
6.1	The Generator’s Goal	13
6.2	Discriminator Loss Function	13
6.2.1	For Real Images ($y = 1$)	13
6.2.2	For Fake/Generated Images ($y = 0$)	13
6.2.3	Combined Discriminator Objective	13
6.3	Generator Loss Function	14
6.4	The Total Combined GAN Objective	14
7	GAN Training Algorithm	15
8	Theoretical Analysis of GANs	16
8.1	The Optimal Discriminator	16
8.2	The Optimal Generator and the JSD Connection	17
8.3	Global Optimality	18
9	Problems, Challenges, and Limitations of GANs	19
9.1	Diminishing Gradients	19
9.2	Nash Equilibrium and Training Instability	19
9.3	Mode Collapse	19
9.4	Summary of GAN Challenges	20

10 Conditional GANs	21
10.1 Conditional GAN Objective	21
10.2 Discriminator Architectures for cGANs	21
10.3 Conditional GAN Training Algorithm	21
10.4 Applications of Conditional GANs	22
10.4.1 Text-to-Image Generation	22
10.4.2 Image-to-Image Translation	22
10.4.3 Speech Enhancement	22
10.4.4 Video Generation	22
11 Case Study: TextureGAN	23
11.1 Overview	23
11.2 Key Design Choices	23
11.3 Loss Functions	23
11.3.1 Ground-Truth Pre-Training Losses	23
11.3.2 External Texture Fine-Tuning Losses	23
11.4 Results	24
12 Summary	25

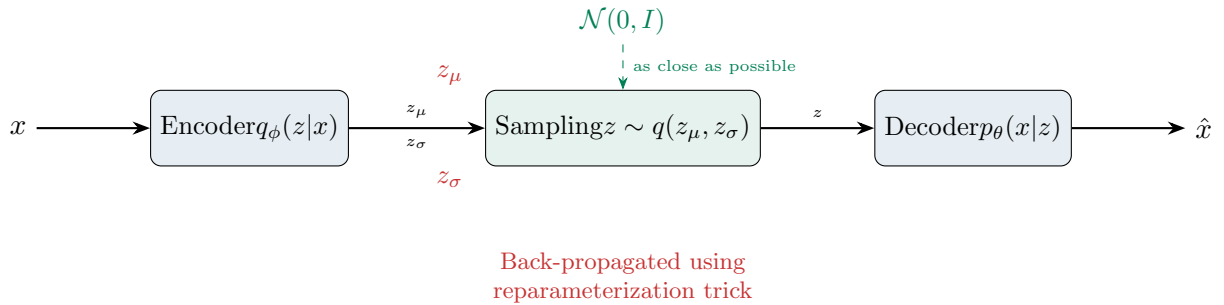
1 Variational AutoEncoder — Summary (Review)

Before diving into Generative Adversarial Networks (GANs), we briefly review the Variational AutoEncoder (VAE) framework, as GANs address several limitations of VAEs.

1.1 Architecture Overview

A VAE consists of two main components:

1. **Encoder (Inference Network)** $q_\phi(z|x)$: A distribution parameterized by ϕ that maps input data x to a latent representation z . The encoder outputs the parameters of the latent distribution — specifically, the mean z_μ and variance z_σ of a Gaussian.
2. **Decoder (Generative Network)** $p_\theta(x|z)$: A distribution parameterized by θ that maps latent codes z back to data space x .



1.2 The ELBO Objective

The VAE is trained by maximizing the **Evidence Lower Bound (ELBO)**:

$$\mathcal{L}(\theta, \phi; x, z) = \underbrace{\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)]}_{\text{Reconstruction Term}} - \underbrace{D_{\text{KL}}(q_\phi(z) \| p(z))}_{\text{KL Divergence Term}} \quad (1)$$

The two terms serve complementary purposes:

- **Reconstruction term** (to be *maximized*): This is the expected log-likelihood of x given z . Maximizing it ensures that the latent codes z sampled from the encoder carry enough information to faithfully reconstruct the original input x . Note that z 's are coming from the encoder, hence this expectation is taken over $q_\phi(z|x)$.
- **KL Divergence term** (to be *minimized*): This forces the learned latent distribution $q_\phi(z)$ to be as close as possible to a known tractable prior $p(z) = \mathcal{N}(0, I)$. Minimizing this KL term regularizes the latent space, preventing the encoder from producing arbitrarily complex distributions.

Key Insight

Maximizing the ELBO simultaneously achieves two goals: good reconstruction quality and a well-structured, smooth latent space. This is what enables VAEs to generate new samples by decoding randomly sampled latent vectors from $\mathcal{N}(0, I)$.

1.3 Loss Functions in VAE

The VAE uses two types of losses:

1. **Reconstruction Loss:** Typically the Mean Squared Error (MSE) or Binary Cross-Entropy between x and $\hat{x}$.
2. **KL Divergence Loss:** Between the encoder's output distribution and the standard normal prior $\mathcal{N}(0, I)$.

The total loss for the VAE is:

$$\text{Loss} = \|x - \hat{x}\| + D_{\text{KL}}\left(q(z_{\mu}, z_{\sigma}) \parallel \mathcal{N}(0, I)\right) \quad (2)$$

While VAEs provide a principled probabilistic framework, they suffer from certain limitations — most notably, the tendency to produce **blurry** generated images. Understanding *why* this happens motivates the transition to GANs.

2 Problems with the L_2 /MSE Loss

2.1 The MSE Loss Function

The basic MSE (Mean Squared Error) or L_2 loss between a reconstructed image $\hat{x}$ and the original image x , each of size $M \times N$, is:

$$\mathcal{L} = \frac{1}{2MN} \sum_{i=1}^M \sum_{j=1}^N (\hat{x}_{ij} - x_{ij})^2 \quad (3)$$

which is proportional to:

$$\mathcal{L} \propto \frac{1}{2} \|\hat{x} - x\|_2^2 \quad (4)$$

2.2 The Perceptual Limitation of MSE

A critical problem with MSE is that it does not align well with human visual perception. Consider an original image and several transformed versions — it is entirely possible to create visually *very different* images that all have the **same MSE** with respect to the original.

Important

MSE treats every pixel independently and weights all pixel errors equally. It does not account for the *structural* or *perceptual* quality of the image. Two images can have identical MSE to a reference but look drastically different to a human observer.

2.3 Connection Between MSE and Gaussian Likelihood

To understand the deeper issue, let us examine the Gaussian distribution:

$$p(x \mid \mu, \sigma^2) = \frac{1}{Z} \exp\left(-\frac{(\mu - x)^2}{2\sigma^2}\right) \quad (5)$$

where $Z = \sqrt{2\pi\sigma^2}$ is the normalization constant, μ is the mean, and σ^2 is the variance. Now, let us set $\mu = \hat{x}$ (the model's prediction/reconstruction) and $\sigma^2 = 1$:

$$p(x \mid \hat{x}) \propto \exp\left(-\frac{1}{2} \|\hat{x} - x\|_2^2\right) \quad (6)$$

Taking the logarithm of both sides:

$$\log p(x \mid \hat{x}) \propto -\frac{1}{2} \|\hat{x} - x\|_2^2 \quad (7)$$

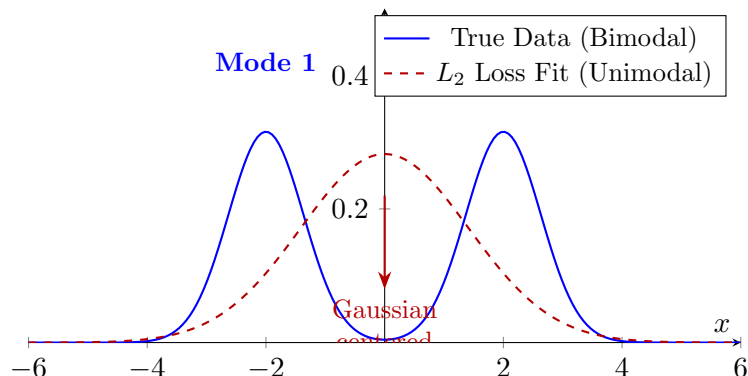
Fundamental Equivalence

Minimizing the L_2 /MSE loss is mathematically equivalent to maximizing the log-likelihood of a Gaussian distribution. This means that when we use MSE as our reconstruction loss, we are implicitly assuming that our real data comes from a *Gaussian* distribution.

2.4 The Blurring Problem: Bimodal Distributions

The Gaussian is a *unimodal* (single-peaked) distribution. However, real-world image distributions are highly complex and **multimodal** — they have many distinct modes (clusters).

What happens when the true data follows a bimodal distribution but we fit a unimodal Gaussian using MSE?



Consider data coming from two distinct modes (e.g., two different types of images, “Red” and “Blue”). The L_2 loss forces the model to minimize reconstruction error for *both* types of samples during training. Since a unimodal Gaussian must be centered *somewhere*, it gets centered right in the **middle** of both modes.

Why VAEs Produce Blurry Images

When we sample from this fitted Gaussian, the samples come from the middle/average of both modes, producing an **average of samples** from the two modes. In image space, this averaging manifests as **blurry images**. The model hedges its bets between multiple plausible outputs, resulting in a fuzzy compromise that does not look like any real image.

3 KL Divergence: Properties and Limitations

Beyond the MSE/reconstruction loss, the other component of the VAE objective is the **KL Divergence**. Understanding its properties and limitations is essential for appreciating why GANs use a different divergence measure.

3.1 Definition and Asymmetry

Definition 3.1: KL Divergence

The **Kullback–Leibler (KL) Divergence** from distribution Q to distribution P is defined as:

$$D_{\text{KL}}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right] \quad (8)$$

For continuous distributions, the sum is replaced by an integral.

Remark

KL Divergence is **not** a true distance metric because it is **not symmetric**:

$$D_{\text{KL}}(P\|Q) \neq D_{\text{KL}}(Q\|P) \quad \text{in general.}$$

This asymmetry has profound implications for how we approximate distributions.

In our setting:

- P = the **target** (unknown distribution we need to estimate)
- Q = the **source** (tractable, parameterized distribution we use to approximate P)

3.2 Forward vs. Reverse KL Divergence

There are two ways to compute the KL divergence between P and Q :

Name	Formula	Behavior
Forward KL	$D_{\text{KL}}(P\ Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$	Zero-avoiding / Mean-seeking
Reverse KL	$D_{\text{KL}}(Q\ P) = \sum_x Q(x) \log \frac{Q(x)}{P(x)}$	Zero-forcing / Mode-seeking

3.3 Why VAEs Use the Reverse KL Divergence

The VAE's KL term is $D_{\text{KL}}(q_\phi(z)\|p(z|x))$, which is the **reverse KL**. Let us understand why.

3.3.1 Forward KL Divergence Analysis

Consider approximating an unknown bimodal distribution P using a parameterized unimodal Gaussian Q_ϕ :

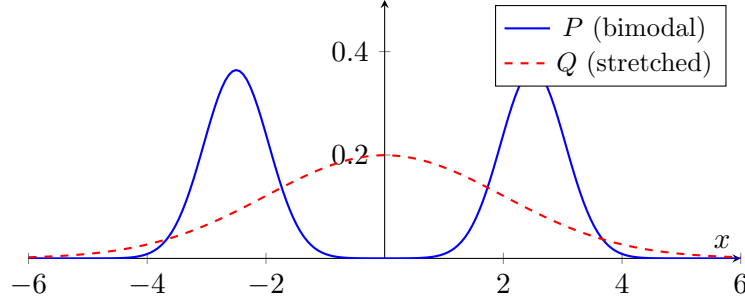
$$\min_{\phi} D_{\text{KL}}(P\|Q_\phi) = \min_{\phi} \sum_x P(x) \log \frac{P(x)}{Q_\phi(x)} \quad (9)$$

Critical question: What happens when $Q(x) = 0$ but $P(x) \neq 0$ (some finite value)?

$$P(x) \log \frac{P(x)}{Q(x)} \rightarrow P(x) \log \frac{P(x)}{0} \rightarrow \infty \quad (10)$$

The forward KL **blows up to infinity!** This means that the forward KL will **never allow** the neural network (parameterized by ϕ) to assign zero probability where P has non-zero probability.

Forward KL: Zero-Avoiding Behavior



Consequence: The forward KL forces Q to *stretch* to cover the **entire** support of P . This leads to:

- **Overestimation:** Q covers regions where P has no mass.
- **Mean-seeking behavior:** Q tries to cover all modes, resulting in a broad, averaged distribution.
- Q assigns probability even in the gap between modes (where there is no data).

3.3.2 Reverse KL Divergence Analysis

Now consider the reverse KL:

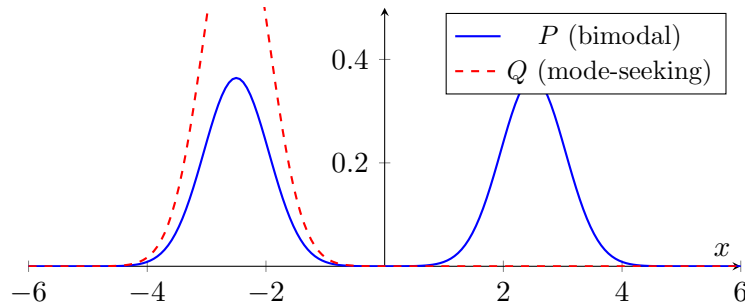
$$\min_{\phi} D_{\text{KL}}(Q_{\phi} \| P) = \min_{\phi} \sum_x Q_{\phi}(x) \log \frac{Q_{\phi}(x)}{P(x)} \quad (11)$$

Critical question: What happens when $Q_{\phi}(x) \neq 0$ but $P(x) = 0$?

$$Q_{\phi}(x) \log \frac{Q_{\phi}(x)}{P(x)} \rightarrow Q_{\phi}(x) \log \frac{Q_{\phi}(x)}{0} \rightarrow \infty \quad (12)$$

The reverse KL blows up! So the optimization will **avoid** placing Q_{ϕ} mass where P is zero.

Reverse KL: Mode-Seeking Behavior



Consequence: The reverse KL forces Q to *shrink* and avoid long tails:

- **Mode-seeking behavior:** Q locks onto one mode of P and tries to match it precisely.
- **Underestimation:** Q may completely miss other modes.
- No probability mass is wasted in regions where P is zero.

Summary: Forward vs. Reverse KL

	Forward KL $D_{\text{KL}}(P\ Q)$	Reverse KL $D_{\text{KL}}(Q\ P)$
Behavior	Stretch Q , support long tails	Shrink Q , avoid long tails
Mode coverage	Mean-seeking (covers all modes)	Mode-seeking (picks one mode)
Estimation	Overestimation	Underestimation

Due to this mode-seeking behavior, **VAEs use the reverse KL**. However, the forward KL (mean-seeking) would introduce even more blur, so neither option is ideal.

3.4 The Fundamental Problem with KL Divergence

Both the forward and reverse KL divergence suffer from an **inherent problem**: they fail to provide a meaningful measure when the two distributions P and Q have **no overlap**.

Non-overlapping Distributions

Consider two discrete distributions:

- P : all mass at $x = 0$, i.e., $P(x = 0) = 1$
- Q : all mass at $x = 0.5$, i.e., $Q(x = 0.5) = 1$

Then:

$$D_{\text{KL}}(P\|Q) = P(x=0) \log \frac{P(x=0)}{Q(x=0)} = 1 \cdot \log \frac{1}{0} = \infty \quad (13)$$

$$D_{\text{KL}}(Q\|P) = Q(x=0.5) \log \frac{Q(x=0.5)}{P(x=0.5)} = 1 \cdot \log \frac{1}{0} = \infty \quad (14)$$

Both KL divergences are **infinite**, providing no useful gradient information!

This is a severe problem in practice: at the **start of optimization**, the approximating distribution Q (which is parameterized and starts randomly) is typically very far from the true distribution P , with little to no overlap. When this happens, the KL divergence is undefined or infinite, making optimization extremely difficult.

4 Jensen–Shannon Divergence

To address the fundamental limitations of KL divergence, we introduce the **Jensen–Shannon Divergence (JSD)**.

4.1 Definition

Definition 4.1: Jensen–Shannon Divergence

The **Jensen–Shannon Divergence** between distributions P and Q is:

$$\text{JSD}(P\|Q) = \frac{1}{2}D_{\text{KL}}(P\|M) + \frac{1}{2}D_{\text{KL}}(Q\|M) \quad (15)$$

where $M = \frac{P+Q}{2}$ is the mixture distribution (average of P and Q).

4.2 Key Properties

- **Symmetric:** $\text{JSD}(P\|Q) = \text{JSD}(Q\|P)$.
- **Bounded:** $0 \leq \text{JSD}(P\|Q) \leq \log 2$.
- **Well-defined for non-overlapping distributions:** Unlike KL, JSD gives a *finite* value even when P and Q have disjoint supports.

4.3 Worked Example: Non-overlapping Distributions

Consider the same non-overlapping distributions from Section 3.4:

- $P: P(x=0) = 1$ $Q: Q(x=0.5) = 1$

The mixture is $M = \frac{P+Q}{2}$, so $M(x=0) = 0.5$ and $M(x=0.5) = 0.5$.

Computing $D_{\text{KL}}(P\|M)$:

$$D_{\text{KL}}(P\|M) = P(x=0) \log \frac{P(x=0)}{M(x=0)} = 1 \cdot \log \frac{1}{0.5} = \log 2 \quad (16)$$

Computing $D_{\text{KL}}(Q\|M)$:

$$D_{\text{KL}}(Q\|M) = Q(x=0.5) \log \frac{Q(x=0.5)}{M(x=0.5)} = 1 \cdot \log \frac{1}{0.5} = \log 2 \quad (17)$$

Therefore:

$$\text{JSD}(P\|Q) = \frac{1}{2}[\log 2 + \log 2] = \log 2 \quad (18)$$

Why JSD Matters for GANs

Unlike KL divergence (which goes to ∞), the JSD gives a **finite value** ($\log 2$) even when P and Q do not overlap at all. This is crucial because at the start of training, the generated distribution p_g is typically very different from the real data distribution p_{data} , and we need a meaningful loss signal to drive optimization.

This motivates the core design of GANs:

1. Instead of MSE, they use **adversarial training**.
2. Instead of KL Divergence, they effectively minimize the **JSD**.

5 From VAEs to GANs: Motivation and Architecture

5.1 Recap: The VAE Framework

In a VAE, the pipeline is:

$$x \xrightarrow{\text{Encoder}} z_\mu, z_\sigma \xrightarrow{\text{Sample from } \mathcal{N}(0, I)} z \xrightarrow{\text{Decoder}} \hat{x}$$

We learn the image distribution $p(x)$ **implicitly** through the latent distribution $p(z)$, which is constrained to be close to $\mathcal{N}(0, I)$. The total VAE loss is:

$$\mathcal{L}_{\text{VAE}} = \|x - \hat{x}\| + D_{\text{KL}}(q(z_\mu, z_\sigma) \parallel \mathcal{N}(0, I)) \quad (19)$$

5.2 The GAN Paradigm Shift

Generative Adversarial Networks (GANs) take a fundamentally different approach:

Aspect	VAE	GAN
Distribution learning	Implicit (via latent space)	Explicit (direct matching)
Reconstruction loss	MSE / L2	Adversarial loss
Divergence measure	KL Divergence	JSD (via adversarial training)
Components	Encoder + Decoder	Generator + Discriminator
Sequence	Encoder $\rightarrow$ Decoder	Decoder(Generator) $\rightarrow$ Encoder(Discriminator)

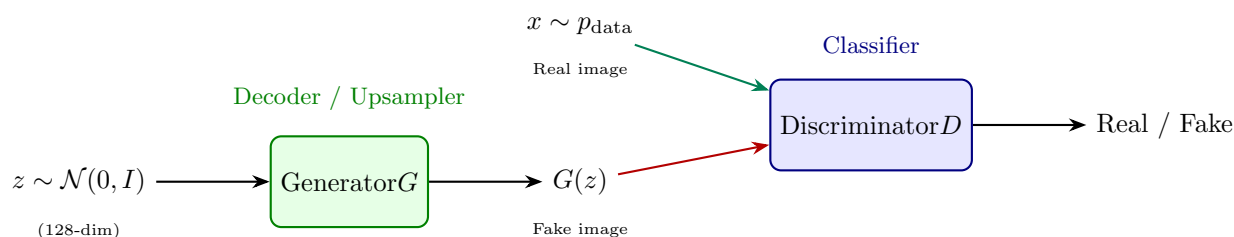
In GANs, the sequence of the decoder (now called the **Generator**) and encoder (now called the **Discriminator**) is **reversed**, and their objectives are tweaked, while keeping the overall goal of generating realistic images.

5.3 GAN Architecture

Definition 5.1: Generative Adversarial Network

A GAN consists of two competing neural networks:

1. **Generator G** : Takes a random noise vector $z \sim p_z(z)$ (e.g., $z \sim \mathcal{N}(0, I)$, dimension ~ 128) and produces a fake image $G(z)$ (e.g., 256×256). This is essentially a deconvolution-based upsampling network (similar to a decoder).
2. **Discriminator D** : Takes an image (real or generated) and outputs a scalar probability indicating whether the image is real or fake. This is a deep classifier network.



The key idea is **adversarial training** — the Generator and Discriminator are in **constant opposition**, competing against each other:

- The Generator's job is to produce images realistic enough to **fool** the Discriminator.
- The Discriminator's job is to correctly **judge** whether an image is real or generated.

Adversarial training is used instead of MSE specifically to **avoid the blurring effect** that MSE introduces. The Discriminator learns to identify the features that distinguish real from fake images, and backpropagates this information to the Generator, helping it improve.

6 GAN Objective Function: Derivation

6.1 The Generator's Goal

Given a random vector z sampled from distribution $p_z(z)$, the Generator produces $G(z)$. The distribution of generated samples is denoted $p_{G(z)}$ or simply p_g .

Objective: Make $p_g(x)$ as close as possible to $p_{\text{data}}(x)$ (the real data distribution).

The Generator works under the **supervision of the Discriminator** — it has no direct access to real data. Instead, it receives feedback through the Discriminator's gradients.

6.2 Discriminator Loss Function

The Discriminator is a binary classifier. Its input is either a real image x or a generated image $G(z)$, and its output $D(\cdot) \in [0, 1]$ represents the probability that the input is real.

Using **binary cross-entropy**:

$$L(\hat{y}, y) = y \log \hat{y} + (1 - y) \log(1 - \hat{y}) \quad (20)$$

where $\hat{y}$ is the predicted label and y is the ground truth label.

6.2.1 For Real Images ($y = 1$)

$$L(D(x), 1) = \log D(x) \quad (21)$$

The Discriminator wants to **maximize** $D(x)$, pushing it towards 1 (correctly classifying real images as real). This means maximizing $\log D(x)$.

6.2.2 For Fake/Generated Images ($y = 0$)

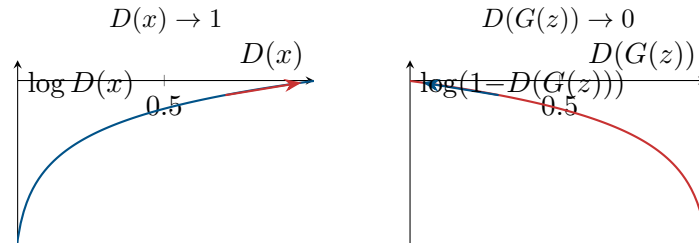
$$L(D(G(z)), 0) = \log [1 - D(G(z))] \quad (22)$$

The Discriminator wants to **minimize** $D(G(z))$ (correctly classifying fake images as fake), which means maximizing $\log[1 - D(G(z))]$.

6.2.3 Combined Discriminator Objective

For a single sample, the Discriminator seeks to:

$$\max_D \left[\log D(x) + \log (1 - D(G(z))) \right] \quad (23)$$



Over a batch, using expectations:

$$\max_D \left[\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))] \right] \quad (24)$$

6.3 Generator Loss Function

The Generator sees only fake data. Its input/output:

- Input to D : $G(z)$ (generated image)
- Output of D : $D(G(z))$ (probability that $G(z)$ is real)
- Ground truth: $y = 0$ (it is fake)

The Generator wants to **fool** the Discriminator, i.e., make $D(G(z))$ as close to 1 as possible. Therefore, from the Generator's perspective:

$$\min_G \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))] \quad (25)$$

Since G never sees real data, the $\mathbb{E}[\log D(x)]$ term is constant with respect to G . For completeness, one can write:

$$\min_G \left[\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))] \right] \quad (26)$$

6.4 The Total Combined GAN Objective

Combining both objectives into a single **minimax game**:

Theorem 6.1: GAN Minimax Objective (Goodfellow et al., 2014)

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))] \quad (27)$$

where the Discriminator D tries to maximize V and the Generator G tries to minimize V .

7 GAN Training Algorithm

The Discriminator and Generator are trained **alternately**: one is updated while the other is held fixed. This alternating optimization is the practical instantiation of the minimax game.

Algorithm 1 Minibatch Stochastic Gradient Descent Training of GANs

- 1: **for** number of training iterations **do**
- 2: — **Train Discriminator (for k steps)** —
- 3: **for** k steps **do**
- 4: Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_z(z)$
- 5: Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data distribution $p_{\text{data}}(x)$
- 6: Update the Discriminator by **ascending** its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right]$$

- 7: **end for**
- 8: — **Train Generator** —
- 9: Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_z(z)$
- 10: Update the Generator by **descending** its stochastic gradient:

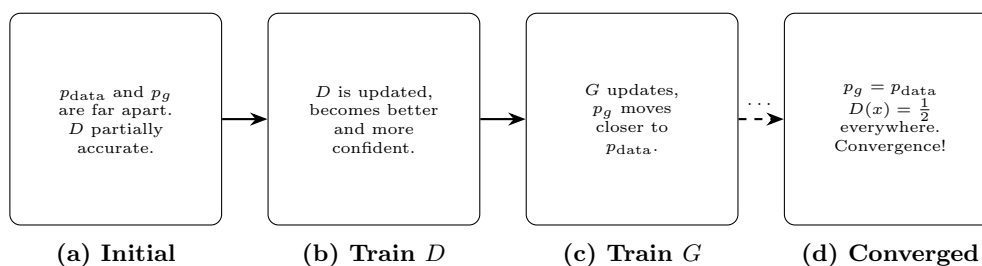
$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)})))$$

- 11: **end for**
-

Remark

The hyperparameter k controls how many Discriminator updates are performed per Generator update. The original paper used $k = 1$ as the least expensive option. The gradient-based updates can use any standard optimizer (the original paper used momentum-based SGD).

Training dynamics visualization (adapted from Goodfellow et al., 2014):



At convergence, the Generator has learned to produce samples indistinguishable from real data, and the Discriminator outputs $D(x) = \frac{1}{2}$ for all inputs (it cannot tell real from fake).

8 Theoretical Analysis of GANs

We now prove two fundamental results from the original GAN paper:

1. For any fixed Generator G , there exists an **optimal Discriminator** D_G^* .
2. For the optimal Discriminator, the optimal Generator minimizes the **JSD** between p_{data} and p_g .

8.1 The Optimal Discriminator

Proposition 8.1: Optimal Discriminator for Fixed G

For any fixed Generator G , the optimal Discriminator D_G^* is:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \quad (28)$$

Proof. For a given G , the Discriminator's objective is:

$$D_G^* = \arg \max_D V(D, G) \quad (29)$$

Step 1: Expand the expectations as integrals.

Recall $\mathbb{E}_{x \sim p}[f(x)] = \int_x f(x) p(x) dx$. Therefore:

$$V(G, D) = \underbrace{\int_x p_{\text{data}}(x) \cdot \log D(x) dx}_{\text{Term } A'} + \underbrace{\int_z p_z(z) \cdot \log (1 - D(G(z))) dz}_{\text{Term } B'} \quad (30)$$

Step 2: Change of variables in Term B' .

Using the random variable transformation: since $x = G(z)$, we have $z = G^{-1}(x)$ (assuming invertibility). By the change of variables formula for densities:

$$p_g(x) = p_z(G^{-1}(x)) \cdot \left| \frac{dG^{-1}(x)}{dx} \right| \quad (31)$$

Substituting $z = G^{-1}(x)$ and $dz = \frac{dG^{-1}(x)}{dx} dx$ into Term B' :

$$\begin{aligned} \int_z p_z(z) \cdot \log (1 - D(G(z))) dz &= \int_x p_z(G^{-1}(x)) \cdot \log (1 - D(x)) \cdot \frac{dG^{-1}(x)}{dx} dx \\ &= \int_x p_g(x) \cdot \log (1 - D(x)) dx \end{aligned} \quad (32)$$

where we used equation (31) in the last step.

Step 3: Combine and write V entirely in terms of x .

$$V(G, D) = \int_x \left[p_{\text{data}}(x) \cdot \log D(x) + p_g(x) \cdot \log (1 - D(x)) \right] dx \quad (33)$$

Both terms are now integrals over x . To maximize V with respect to $D(x)$, we can maximize the integrand pointwise.

Step 4: Differentiate with respect to $D(x)$ and set to zero.

For fixed x , define $f(d) = a \log d + b \log(1 - d)$ where $a = p_{\text{data}}(x)$ and $b = p_g(x)$.

$$\frac{df}{d[D(x)]} = \frac{p_{\text{data}}(x)}{D(x)} - \frac{p_g(x)}{1 - D(x)} = 0 \quad (34)$$

Solving:

$$p_{\text{data}}(x) \cdot (1 - D(x)) = p_g(x) \cdot D(x) \quad (35)$$

$$p_{\text{data}}(x) = D(x) \cdot (p_{\text{data}}(x) + p_g(x)) \quad (36)$$

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \quad (37)$$

□

Remark

While this formula is elegant, it **cannot be computed in practice** because we do not know $p_{\text{data}}(x)$ or $p_g(x)$ analytically. The neural network D_θ approximates this optimal discriminator through training.

8.2 The Optimal Generator and the JSD Connection

Now, given the optimal Discriminator D_G^* , we seek the optimal Generator:

$$G^* = \arg \min_G V(D_G^*, G) \quad (38)$$

Theorem 8.1: GAN Objective Equals JSD

For the optimal Discriminator D_G^* , the value function becomes:

$$V(D_G^*, G) = -\log 4 + 2 \cdot \text{JSD}(p_{\text{data}} \| p_g) \quad (39)$$

Therefore:

$$G^* = \arg \min_G \left[-\log 4 + 2 \cdot \text{JSD}(p_{\text{data}} \| p_g) \right] \quad (40)$$

Proof. Substituting $D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$ into $V(G, D)$:

$$V(D_G^*, G) = \int_x p_{\text{data}}(x) \log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} dx + \int_x p_g(x) \log \frac{p_g(x)}{p_{\text{data}}(x) + p_g(x)} dx \quad (41)$$

Now we add and subtract $\log 2$ terms strategically. Note that:

$$\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} = \log \frac{2 \cdot p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} - \log 2 \quad (42)$$

Substituting:

$$V(D_G^*, G) = \int_x p_{\text{data}}(x) \left[\log \frac{2 \cdot p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} - \log 2 \right] dx + \int_x p_g(x) \left[\log \frac{2 \cdot p_g(x)}{p_{\text{data}}(x) + p_g(x)} - \log 2 \right] dx \quad (43)$$

Distributing and using $\int p_{\text{data}}(x) dx = \int p_g(x) dx = 1$:

$$\begin{aligned}
V(D_G^*, G) &= -\log 2 \cdot \underbrace{\int_x p_{\text{data}}(x) dx}_{=1} - \log 2 \cdot \underbrace{\int_x p_g(x) dx}_{=1} \\
&\quad + \int_x p_{\text{data}}(x) \log \frac{2 \cdot p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} dx + \int_x p_g(x) \log \frac{2 \cdot p_g(x)}{p_{\text{data}}(x) + p_g(x)} dx \quad (44)
\end{aligned}$$

Let $M(x) = \frac{p_{\text{data}}(x) + p_g(x)}{2}$. Then $\frac{2 \cdot p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} = \frac{p_{\text{data}}(x)}{M(x)}$, so:

$$\begin{aligned}
V(D_G^*, G) &= -2 \log 2 + \underbrace{\int_x p_{\text{data}}(x) \log \frac{p_{\text{data}}(x)}{M(x)} dx}_{D_{\text{KL}}(p_{\text{data}} \| M)} + \underbrace{\int_x p_g(x) \log \frac{p_g(x)}{M(x)} dx}_{D_{\text{KL}}(p_g \| M)} \quad (45)
\end{aligned}$$

Recognizing that $\text{JSD}(p_{\text{data}} \| p_g) = \frac{1}{2} [D_{\text{KL}}(p_{\text{data}} \| M) + D_{\text{KL}}(p_g \| M)]$:

$$\boxed{V(D_G^*, G) = -\log 4 + 2 \cdot \text{JSD}(p_{\text{data}} \| p_g)} \quad (46)$$

□

8.3 Global Optimality

Theorem 8.2: Global Minimum of the GAN Objective

The global minimum of $V(D_G^*, G)$ is achieved if and only if $p_g = p_{\text{data}}$, and the minimum value is $-\log 4$.

Proof. Since $\text{JSD}(p_{\text{data}} \| p_g) \geq 0$ with equality if and only if $p_{\text{data}} = p_g$, the minimum of $-\log 4 + 2 \cdot \text{JSD}(p_{\text{data}} \| p_g)$ is achieved when $\text{JSD} = 0$, i.e., when $p_g = p_{\text{data}}$. □

At convergence:

- $\text{JSD} \rightarrow 0$
- $p_g(x) \rightarrow p_{\text{data}}(x)$ (the generator perfectly matches the data distribution)
- $V(D, G) \rightarrow -\log 4$
- $D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} = \frac{p_{\text{data}}(x)}{2 \cdot p_{\text{data}}(x)} = \frac{1}{2}$ for all x

The Elegant Result

The GAN minimax game is equivalent to minimizing the Jensen–Shannon Divergence between the generated distribution and the real data distribution. At the optimal solution, the generator’s distribution exactly matches the data distribution, and the discriminator can no longer distinguish real from fake (outputting $\frac{1}{2}$ everywhere).

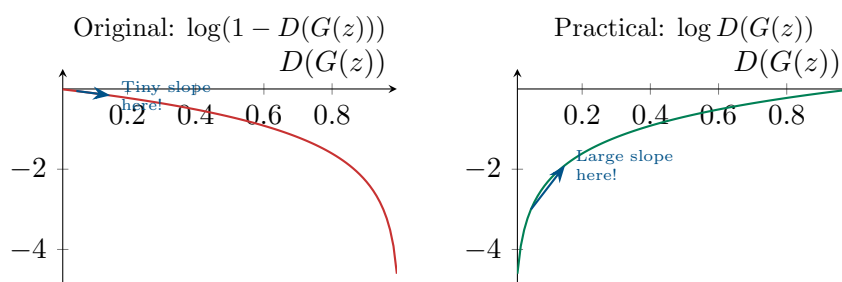
9 Problems, Challenges, and Limitations of GANs

Despite the elegant theoretical framework, training GANs in practice is notoriously difficult. The main challenges include:

9.1 Diminishing Gradients

Initially, the Generator produces very poor samples that are easy for the Discriminator to classify. The Discriminator trains quickly and becomes very confident ($D(x) \approx 1$ for real, $D(G(z)) \approx 0$ for fake).

The problem: The Generator's loss $\log(1 - D(G(z)))$ has a **very small gradient** when $D(G(z)) \approx 0$:



Practical fix: Instead of minimizing $\log(1 - D(G(z)))$, the Generator maximizes $\log D(G(z))$:

$$\min_G \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad \longleftrightarrow \quad \max_G \mathbb{E}_{z \sim p_z} [\log D(G(z))] \quad (47)$$

This provides much stronger gradients early in training when $D(G(z)) \approx 0$.

9.2 Nash Equilibrium and Training Instability

The GAN is a **zero-sum, non-cooperative game**. It converges only when D and G reach some equilibrium (Nash equilibrium).

Definition 9.1: Nash Equilibrium

A **Nash equilibrium** is a state where no player can improve their outcome by unilaterally changing their strategy, given the other player's strategy. In a Nash equilibrium, each player's strategy is optimal given the other's.

In practice, both G and D are trained using gradient descent, which is not guaranteed to converge to a Nash equilibrium in non-convex games. The training often exhibits **oscillatory behavior** rather than smooth convergence.

9.3 Mode Collapse

Mode collapse occurs when the Generator learns to produce only a limited variety of outputs (covering only some modes of the data distribution), rather than the full diversity of real data. For example, a GAN trained on handwritten digits might only generate “1”s and “7”s, ignoring other digits entirely.

9.4 Summary of GAN Challenges

- (a) **Vanishing Gradient:** Discriminator becomes too strong early on.
- (b) **Mode Collapse:** Generator produces limited variety.
- (c) **Achieving Equilibrium:** No guarantee of convergence in practice.
- (d) **Counting Objects / Object Semantics:** GANs struggle with understanding high-level semantics (e.g., generating the right number of objects).

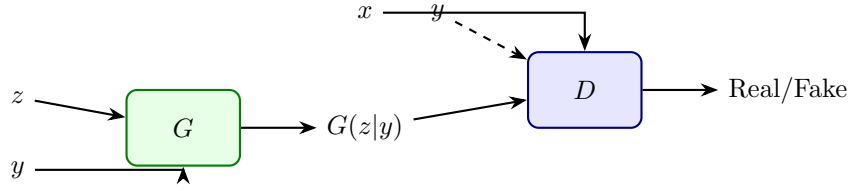
10 Conditional GANs

Standard GANs generate images without any control over the content. **Conditional GANs (cGANs)** extend the framework by conditioning the generation on additional information y (e.g., class labels, text descriptions, or other images).

10.1 Conditional GAN Objective

The standard GAN objective is modified by conditioning both G and D on y :

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x|y)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z|y)))] \quad (48)$$



The condition y is fed to **both** the Generator and the Discriminator. The Discriminator now has two tasks:

1. Judge whether the image x is realistic (real vs. fake).
2. Judge whether x matches the condition y (condition matching).

10.2 Discriminator Architectures for cGANs

Two common designs for the conditional Discriminator:

1. **Joint evaluation:** Both x and c are processed through separate networks and then merged. The combined network outputs a single score for both realism and condition-matching.
2. **Separate heads:** The image x is processed by one network that evaluates realism, while a second network evaluates whether c and x are matched. This decomposition has been explored in works by Odena et al. (ICML, 2017), Miyato et al. (ICLR, 2018), and Zhang et al. (arXiv, 2017).

10.3 Conditional GAN Training Algorithm

In each training iteration:

Learning D :

1. Sample m positive examples $\{(c^i, x^i)\}$ from the database.
2. Sample m noise vectors $\{z^i\}$ and generate $\{\hat{x}^i = G(c^i, z^i)\}$.
3. Sample m mismatched objects $\{\hat{x}^i\}$ from the database (for condition matching).
4. Update θ_d to maximize:

$$\tilde{V} = \frac{1}{m} \sum_{i=1}^m \log D(c^i, x^i) + \frac{1}{m} \sum_{i=1}^m \log (1 - D(c^i, \hat{x}^i)) + \frac{1}{m} \sum_{i=1}^m \log (1 - D(c^i, \hat{x}^i)) \quad (49)$$

Learning G :

1. Sample m noise vectors and conditions.
2. Update θ_g to maximize: $\tilde{V} = \frac{1}{m} \sum_{i=1}^m \log D(G(c^i, z^i))$

10.4 Applications of Conditional GANs

10.4.1 Text-to-Image Generation

Conditional GANs enable generating images from text descriptions. The traditional supervised approach (text $\rightarrow$ NN $\rightarrow$ image, trained with MSE) suffers from blurriness because the same text can correspond to many different valid images — the network learns the average.

By conditioning a GAN on text embeddings, the Generator learns to produce diverse, sharp images consistent with the description. Notable work: **StackGAN** (Zhang et al., ICCV 2017) uses a two-stage approach with conditioning augmentation to generate high-resolution (256×256) images from text.

10.4.2 Image-to-Image Translation

The condition can also be an image itself, enabling tasks like:

- Semantic labels $\rightarrow$ street scenes
- Architectural labels $\rightarrow$ building facades
- Black-and-white $\rightarrow$ color images
- Aerial photos $\rightarrow$ maps
- Day $\rightarrow$ night images
- Edge maps $\rightarrow$ photo-realistic images

The seminal work **pix2pix** (Isola et al., CVPR 2017) demonstrated these transformations using a conditional GAN with a U-Net generator and a **PatchGAN** discriminator (which evaluates image patches rather than the entire image, enforcing local realism).

10.4.3 Speech Enhancement

Conditional GANs can also be applied to audio: given a noisy spectrogram, the Generator produces a cleaned spectrogram, and the Discriminator judges whether a (noisy, clean) pair looks authentic.

10.4.4 Video Generation

Given a sequence of frames, the Generator predicts the next frame. The Discriminator receives the sequence plus the generated/real final frame and determines whether it is real or generated.

11 Case Study: TextureGAN

TextureGAN (presented in the context of “Deep Learning and its Applications”, CS-671) demonstrates how conditional GANs can be used for texture-controlled image synthesis.

11.1 Overview

TextureGAN generates novel instances of common items (handbags, shoes, clothing) from **sketches** and simple **texture patches**. A user places a small texture patch inside a sketch, and the network propagates that texture across the entire object.

11.2 Key Design Choices

- **LAB color space:** The authors work in LAB rather than RGB. The L channel captures luminance (structure/texture), while the AB channels capture color. This separation allows applying different losses to structure vs. color.
- **5-channel input:** One channel for sketch, two channels for texture (intensity + binary mask), and two channels for color.
- **Two-stage training:** First, ground-truth pre-training with known textures; then, external texture fine-tuning for unseen textures.

11.3 Loss Functions

11.3.1 Ground-Truth Pre-Training Losses

1. **Feature Loss ($\mathcal{L}_F$):** L2 loss on deeper VGG-19 features (L channel only) to capture structural similarity.
2. **Adversarial Loss ($\mathcal{L}_{ADV}$):** Applied only on the L channel (grayscale) to encourage sharp, realistic textures without constraining color.
3. **Style Loss ($\mathcal{L}_S$):** Matches Gram matrices of VGG-19 features (L channel) between generated and ground-truth images, capturing fine-scale texture details.
4. **Pixel Loss ($\mathcal{L}_P$):** Weak L2 pixel loss on L channel to stabilize training.
5. **Color Loss ($\mathcal{L}_C$):** L2 loss on AB channels for color fidelity.

Overall pre-training objective:

$$\mathcal{L} = \mathcal{L}_F + w_{ADV}\mathcal{L}_{ADV} + w_S\mathcal{L}_S + w_P\mathcal{L}_P + w_C\mathcal{L}_C \quad (50)$$

11.3.2 External Texture Fine-Tuning Losses

For unseen textures (no ground truth), local texture losses are added:

1. **Local Texture Loss ($\mathcal{L}_t$):** Randomly sampled patches from the generated image are compared to input texture patches using: $\mathcal{L}_t = \mathcal{L}_s + w_p\mathcal{L}_p + w_{adv}\mathcal{L}_{adv}$ (local style, pixel, and adversarial losses).
2. **Local Adversarial Loss:** Determines whether generated patches have the same texture as input patches.

Overall fine-tuning objective:

$$\mathcal{L} = \mathcal{L}_F + w_{ADV}\mathcal{L}_{ADV} + w_P\mathcal{L}'_P + w_C\mathcal{L}'_C + \mathcal{L}_t \quad (51)$$

11.4 Results

TextureGAN demonstrates effective texture propagation from a small patch to an entire object. The ablation studies show that removing the adversarial loss leads to blurry results, and removing the texture/style loss harms texture propagation — confirming the complementary roles of these loss components.

12 Summary

This set of lecture notes covered the journey from Variational AutoEncoders to Generative Adversarial Networks:

1. **VAE Review:** Encoder-decoder architecture with ELBO objective (reconstruction + KL divergence).
2. **Problems with MSE:** $\text{MSE} \equiv \text{Gaussian log-likelihood} \Rightarrow \text{assumes unimodal data} \Rightarrow \text{blurry outputs for multimodal data}.$
3. **KL Divergence Limitations:** Forward KL is mean-seeking (overestimation); Reverse KL is mode-seeking (underestimation). Both fail when distributions do not overlap.
4. **JS Divergence:** Symmetric, bounded ($[0, \log 2]$), and well-defined even for non-overlapping distributions.
5. **GAN Framework:** Generator (G) and Discriminator (D) in a minimax game. Adversarial training replaces MSE; JSD replaces KL divergence.
6. **GAN Objective:** $\min_G \max_D V(D, G) = \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$
7. **Theoretical Results:**
 - Optimal Discriminator: $D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$
 - Optimal Generator minimizes $\text{JSD}(p_{\text{data}} \| p_g)$
 - Global minimum at $p_g = p_{\text{data}}, V^* = -\log 4, D^* = \frac{1}{2}$
8. **Practical Challenges:** Diminishing gradients, mode collapse, Nash equilibrium instability.
9. **Conditional GANs:** Condition on labels/text/images for controlled generation. Applications: text-to-image, image-to-image, speech enhancement, video generation.
10. **TextureGAN:** Texture-controlled image synthesis using sketch + texture patches with multiple loss functions (feature, adversarial, style, pixel, color, local texture).

The Big Picture

GANs represent a paradigm shift in generative modeling: instead of explicitly defining and optimizing a likelihood function (which requires tractable density assumptions like Gaussians), GANs use an *adversarial game* that implicitly minimizes the Jensen–Shannon Divergence between the generated and real data distributions. This avoids the restrictive assumptions that cause blurriness in VAEs, at the cost of more challenging training dynamics.

References

1. Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). *Generative Adversarial Nets*. Advances in Neural Information Processing Systems (NeurIPS).
2. Kingma, D. P. & Welling, M. (2014). *Auto-Encoding Variational Bayes*. International Conference on Learning Representations (ICLR).
3. Mirza, M. & Osindero, S. (2014). *Conditional Generative Adversarial Nets*. arXiv:1411.1784.
4. Isola, P., Zhu, J.-Y., Zhou, T., & Efros, A. A. (2017). *Image-to-Image Translation with Conditional Adversarial Networks*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
5. Zhang, H., Xu, T., Li, H., Zhang, S., Wang, X., Huang, X., & Metaxas, D. (2017). *StackGAN: Text to Photo-realistic Image Synthesis with Stacked Generative Adversarial Networks*. IEEE International Conference on Computer Vision (ICCV).
6. Reed, S., Akata, Z., Yan, X., Logeswaran, L., Schiele, B., & Lee, H. (2016). *Generative Adversarial Text to Image Synthesis*. International Conference on Machine Learning (ICML).
7. Xian, W., Sangkloy, P., Agrawal, V., Raj, A., Lu, J., Fang, C., Yu, F., & Hays, J. (2018). *TextureGAN: Controlling Deep Image Synthesis with Texture Patches*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
8. Odena, A., Olah, C., & Shlens, J. (2017). *Conditional Image Synthesis with Auxiliary Classifier GANs*. International Conference on Machine Learning (ICML).